STAGE 2: TESTING & QA

Complete Testing Guide - All Issues, Fixes, and Checklists

Project: [YOUR PROJECT NAME]
Timeline: Day 4 (8 hours)
Goal: Bug-free build ready for deployment

SECTION 1: TESTING OVERVIEW

1.1 Testing Phases

Phase 1: Unit Testing (2 hours)

- Test individual functions
- Test edge cases
- Target: 100% coverage on critical paths

Phase 2: Integration Testing (2 hours)

- Test API endpoints
- Test database operations
- Test third-party integrations

Phase 3: E2E Testing (2 hours)

- Test critical user flows
- Test on real devices

- Test all platforms

Phase 4: Performance & Security (2 hours)

- Performance testing
- Security testing
- Bug triage & fixing

1.2 Success Criteria

- [ ] All unit tests passing
- [ ] All integration tests passing
- [ ] All E2E tests passing
- [ ] <1% error rate
- [ ] No critical bugs
- [ ] Performance acceptable
- [ ] Security verified

SECTION 2: UNIT TESTING (2 hours)

2.1 Run Unit Tests

Step 1: Run All Tests

```
npm run test
npm run test:coverage
```

Step 2: Check Coverage

- [ ] 100% coverage on critical paths

- [ ] >80% overall coverage

Step 3: Fix Failures

```
npm run test:watch
# Fix failing tests one by one
```

2.2 Common Unit Test Issues

Issue 1: Test Timeout

Problem: Test takes too long

```
FAIL  src/features/invoice.test.ts
  ● Invoice Feature › should create invoice
    Timeout - Async callback was not invoked within the 5000ms timeout
```

Fix:

```
// Increase timeout
test('should create invoice', async () => {
  // ...
}, 10000);  // 10 second timeout

// OR mock slow operations
jest.mock('axios', () => ({
  post: jest.fn().mockResolvedValue({ data: {} })
}));
```

Issue 2: Cannot Find Module

Problem: Import path wrong

```
FAIL  src/features/invoice.test.ts
  ● Cannot find module '../utils/helpers' from
'src/features/invoice.test.ts'
```

Fix:

```
// Check import path
import { helper } from '../utils/helpers';  // Correct
// NOT: import { helper } from '../utils/helper';  // Wrong
```

Issue 3: Undefined Variable

Problem: Variable not initialized

```
FAIL  src/features/invoice.test.ts
  ● TypeError: Cannot read property 'id' of undefined
```

Fix:

```
// Initialize before use
let invoice;

beforeEach(() => {
  invoice = { id: 1, amount: 100 };
});

test('should access invoice id', () => {
  expect(invoice.id).toBe(1);
});
```

SECTION 3: INTEGRATION TESTING (2 hours)

3.1 Test API Endpoints

Step 1: Start Server

```
npm run dev
# Server running on http://localhost:5000
```

Step 2: Test GET Endpoints

```
curl http://localhost:5000/api/invoices
# Should return: { data: [...], status: 200 }
```

Step 3: Test POST Endpoints

```
curl -X POST http://localhost:5000/api/invoices \
  -H "Content-Type: application/json" \
  -d '{"amount": 100, "client": "John"}'
# Should return: { data: { id: 1, ... }, status: 201 }
```

Step 4: Test Error Responses

```
curl http://localhost:5000/api/invoices/999
# Should return: { error: "Not found", status: 404 }
```

3.2 Common Integration Test Issues

Issue 1: Database Connection Failed

Problem: Cannot connect to database

```
Error: ECONNREFUSED 127.0.0.1:5432
Database connection failed
```

Fix:

```
# Check database is running
psql -U postgres -d yourdb

# OR check connection string
echo $DATABASE_URL
# Should be: postgresql://user:pass@localhost:5432/db

# OR restart database
docker-compose up -d postgres
```

Issue 2: API Timeout

Problem: API request takes too long

```
Error: Request timeout after 5000ms
```

Fix:

```
// Increase timeout
const response = await axios.get(url, { timeout: 10000 });

// OR optimize query
// Add database indexes
// Add caching
// Paginate results
```

Issue 3: Payment Gateway Error

Problem: Payment integration fails

```
Error: Invalid API key
Stripe API key not found
```

Fix:

```
# Check API key in .env
cat .env | grep STRIPE_SECRET_KEY

# Should be: STRIPE_SECRET_KEY=sk_test_...

# OR use test key
STRIPE_SECRET_KEY=sk_test_4eC39HqLyjWDarhtT657tB42
```

SECTION 4: E2E TESTING (2 hours)

4.1 Test Critical User Flows

Flow 1: User Signup

Steps:

1. Open app
2. Click “Sign Up”
3. Enter email: test@example.com
4. Enter password: Test123!
5. Click “Sign Up”

Expected:

- [ ] User created
- [ ] Email verified
- [ ] Redirected to dashboard
- [ ] User logged in

Test on:

- [ ] Web (Chrome, Safari, Firefox)
- [ ] iOS (iPhone 12, 14, 15)
- [ ] Android (Pixel, Galaxy)
- [ ] Tablet (iPad, Galaxy Tab)

Flow 2: Create Invoice

Steps:

1. Login
2. Click “New Invoice”
3. Select client
4. Enter amount: $100
5. Click “Send”

Expected:

- [ ] Invoice created
- [ ] Invoice sent
- [ ] Email sent to client

- [ ] Confirmation message shown

Test on:

- [ ] All platforms
- [ ] Different networks (WiFi, 4G, offline)

Flow 3: Payment Processing

Steps:

1. Client receives invoice email
2. Client clicks “Pay”
3. Client enters card details
4. Client clicks “Pay Now”

Expected:

- [ ] Payment processed
- [ ] Invoice marked as paid
- [ ] Confirmation email sent
- [ ] Receipt generated

Test on:

- [ ] All platforms
- [ ] Test card: 4242 4242 4242 4242

SECTION 5: PERFORMANCE TESTING (1 hour)

5.1 Page Load Time

Target: <3 seconds

Test:

```
# Use Lighthouse
npm install -g lighthouse
lighthouse https://yourapp.com --view

# OR use WebPageTest
# Visit: https://www.webpagetest.org
# Enter URL and run test
```

Common Issues:

- Large images → Compress/optimize
- Large bundle → Code split
- Slow API → Add caching
- Slow database → Add indexes

5.2 API Response Time

Target: <500ms

Test:

```
# Use curl with timing
curl -w "@curl-format.txt" -o /dev/null -s https://yourapp.com/api/invoices

# OR use Apache Bench
ab -n 100 -c 10 https://yourapp.com/api/invoices
```

Common Issues:

- Slow query → Add indexes
- N+1 problem → Optimize query
- Missing cache → Add caching
- Inefficient code → Optimize algorithm

5.3 Bundle Size

Target: <500KB

Test:

```
npm run build
# Check dist/ folder size

# OR use webpack-bundle-analyzer
npm install --save-dev webpack-bundle-analyzer
npm run build:analyze
```

Common Issues:

- Large dependency → Find alternative
- Duplicate code → Remove duplication
- Unused code → Remove or tree-shake
- Large assets → Compress/optimize

SECTION 6: SECURITY TESTING (1 hour)

6.1 Input Validation

Test: Can user break app with invalid input?

Test Cases:

- [ ] Empty input: “”
- [ ] Very long input: “a” * 10000
- [ ] Special characters: “”
- [ ] SQL injection: “’ OR ‘1’=‘1”
- [ ] Null/undefined: null, undefined
- [ ] Wrong type: “abc” for number field

Expected: App handles gracefully, shows error message

6.2 Authentication

Test: Can user access without login?

Test Cases:

- [ ] Access /dashboard without token
- [ ] Use invalid token
- [ ] Use expired token
- [ ] Use token from different user

Expected: User redirected to login

6.3 Authorization

Test: Can user access other user’s data?

Test Cases:

- [ ] User A tries to access User B's invoices
- [ ] User A tries to delete User B's invoice
- [ ] User A tries to edit User B's settings

Expected: Access denied, 403 error

6.4 Payment Security

Test: Is payment data secure?

Test Cases:

- [ ] Card number not logged
- [ ] Card number not in database
- [ ] Payment uses HTTPS
- [ ] Webhook signature verified

Expected: All security checks pass

SECTION 7: BUG TRIAGE & FIXING (1 hour)

7.1 Bug Classification

Critical (Fix immediately):

- App crashes
- Data loss
- Security vulnerability
- Payment failure

High (Fix before launch):

- Feature doesn't work
- UI broken
- Performance issue
- User confusion

Medium (Fix in next release):

- Minor UI issue
- Typo
- Edge case
- Non-critical feature

Low (Fix later):

- Nice to have
- Enhancement
- Polish

7.2 Bug Fixing Process

Step 1: Reproduce Bug

- [ ] Can you reproduce it?
- [ ] On which platform?
- [ ] What steps?

Step 2: Find Root Cause

- [ ] Check logs
- [ ] Debug code
- [ ] Check database
- [ ] Check API

Step 3: Fix Bug

- [ ] Write test first
- [ ] Implement fix
- [ ] Verify fix works
- [ ] Verify no regressions

Step 4: Commit Fix

```
git add .
git commit -m "fix: [bug description]"
```

SECTION 8: TESTING CHECKLISTS

8.1 Pre-Launch Testing Checklist

Functionality

- [ ] All P0 features work
- [ ] All P1 features work
- [ ] Payment processing works
- [ ] Email notifications work
- [ ] Error handling works

Performance

- [ ] Page load time <3s
- [ ] API response time <500ms
- [ ] Bundle size <500KB
- [ ] No memory leaks
- [ ] No CPU spikes

Security

- [ ] Input validation works
- [ ] Authentication works
- [ ] Authorization works
- [ ] Payment is secure
- [ ] No hardcoded credentials

Compatibility

- [ ] Works on iOS 13+
- [ ] Works on Android 8+
- [ ] Works on Chrome, Safari, Firefox
- [ ] Works on mobile, tablet, desktop
- [ ] Works on 4G, WiFi, offline

User Experience

- [ ] UI responsive
- [ ] Touch targets 44x44px
- [ ] Font size readable
- [ ] Color contrast WCAG AA
- [ ] Loading states visible
- [ ] Error messages clear
- [ ] Success messages visible

Business

- [ ] Pricing correct
- [ ] Payment gateway working
- [ ] Invoices generated correctly
- [ ] Reports accurate

- [ ] Admin dashboard functional

8.2 Real Device Testing Checklist

iOS

- [ ] iPhone 12 (iOS 16)
- [ ] iPhone 14 (iOS 17)
- [ ] iPhone 15 (iOS 18)
- [ ] iPad (iOS 17)

Android

- [ ] Samsung Galaxy S21 (Android 12)
- [ ] Samsung Galaxy S23 (Android 13)
- [ ] Google Pixel 6 (Android 13)
- [ ] Google Pixel 7 (Android 14)

Web

- [ ] Chrome (latest)
- [ ] Safari (latest)
- [ ] Firefox (latest)
- [ ] Edge (latest)

Networks

- [ ] WiFi (fast)
- [ ] 4G (medium)
- [ ] 3G (slow)
- [ ] Offline (no network)

SECTION 9: COMMON BUGS & FIXES

9.1 UX/UI Bugs

Bug 1: Text Overlapping

Symptom: Text overlaps on mobile **Cause**: Fixed width, not responsive **Fix**:

```
/* Before */
.button { width: 200px; }

/* After */
.button {
  max-width: 100%;
  width: 200px;
  @media (max-width: 480px) {
    width: 100%;
  }
}
```

Bug 2: Button Not Clickable

Symptom: Button doesn't respond to clicks **Cause**: Touch target too small **Fix**:

```
/* Before */
.button { padding: 5px 10px; }  /* 15x30px */

/* After */
.button { padding: 12px 16px; }  /* 44x44px minimum */
```

Bug 3: Lag on Scroll

Symptom: App stutters when scrolling **Cause**: Heavy rendering in scroll handler **Fix**:

```
// Before
window.addEventListener('scroll', () => {
  // Heavy calculation
  updateDOM();
});

// After - Use passive listener
window.addEventListener('scroll', () => {
  updateDOM();
}, { passive: true });

// OR use requestAnimationFrame
let ticking = false;
window.addEventListener('scroll', () => {
  if (!ticking) {
    requestAnimationFrame(() => {
      updateDOM();
      ticking = false;
    });
    ticking = true;
  }
});
```

9.2 Data Bugs

Bug 1: Data Not Saving

Symptom: User creates invoice, but it doesn't save **Cause**: API error not handled **Fix**:

```
// Before
async function createInvoice(data) {
  const response = await fetch('/api/invoices', {
    method: 'POST',
    body: JSON.stringify(data)
  });
  return response.json();
}

// After - Add error handling
async function createInvoice(data) {
  try {
    const response = await fetch('/api/invoices', {
      method: 'POST',
      body: JSON.stringify(data)
    });
    if (!response.ok) {
      throw new Error(`API error: ${response.status}`);
    }
    return response.json();
  } catch (error) {
    console.error('Failed to create invoice:', error);
    showErrorMessage('Failed to create invoice. Please try again.');
    throw error;
  }
}
```

Bug 2: Duplicate Data

Symptom: User clicks “Save” twice, creates 2 invoices **Cause**: No idempotency **Fix**:

```
// Before
async function createInvoice(data) {
  const response = await fetch('/api/invoices', {
    method: 'POST',
    body: JSON.stringify(data)
  });
  return response.json();
}

// After - Add idempotency key
async function createInvoice(data) {
  const idempotencyKey = generateUUID();
  const response = await fetch('/api/invoices', {
    method: 'POST',
    headers: {
      'Idempotency-Key': idempotencyKey
    },
    body: JSON.stringify(data)
  });
  return response.json();
}
```

9.3 Payment Bugs

Bug 1: Payment Fails Silently

Symptom: User clicks “Pay”, nothing happens **Cause**: Error not caught **Fix**:

```
// Before
async function processPayment(amount) {
  const paymentIntent = await stripe.paymentIntents.create({
    amount: amount * 100,
    currency: 'usd'
  });
  return paymentIntent;
}

// After - Add error handling
async function processPayment(amount) {
  try {
    const paymentIntent = await stripe.paymentIntents.create({
      amount: amount * 100,
      currency: 'usd'
    });
    return paymentIntent;
  } catch (error) {
    console.error('Payment failed:', error);
    showErrorMessage('Payment failed. Please try again.');
    throw error;
  }
}
```

Bug 2: Webhook Not Received

Symptom: Payment succeeds but invoice not marked as paid **Cause**: Webhook signature not verified **Fix**:

```
// Before
app.post('/webhook', (req, res) => {
  const event = req.body;
  handlePaymentEvent(event);
  res.json({ received: true });
});

// After - Verify signature
app.post('/webhook', (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;

  try {
    event = stripe.webhooks.constructEvent(
      req.rawBody,
      sig,
      process.env.STRIPE_WEBHOOK_SECRET
    );
  } catch (error) {
    console.error('Webhook signature verification failed:', error);
    return res.status(400).send(`Webhook Error: ${error.message}`);
  }

  handlePaymentEvent(event);
  res.json({ received: true });
});
```

CONCLUSION

Testing stage complete when:

- ✅ All unit tests passing
- ✅ All integration tests passing
- ✅ All E2E tests passing
- ✅ <1% error rate
- ✅ No critical bugs
- ✅ Performance acceptable

- ✅ Security verified

Next Step: Move to STAGE 3 - DEPLOYMENT PDF

Document Version: 1.0
Created: [DATE]
Author: [YOUR NAME]
Status: [Draft / Active]